#Hashlock.

Pool Party (dApp)

SMART CONTRACT

Security Audit

Platform

ETH

hashlock.com.au

May 2024

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

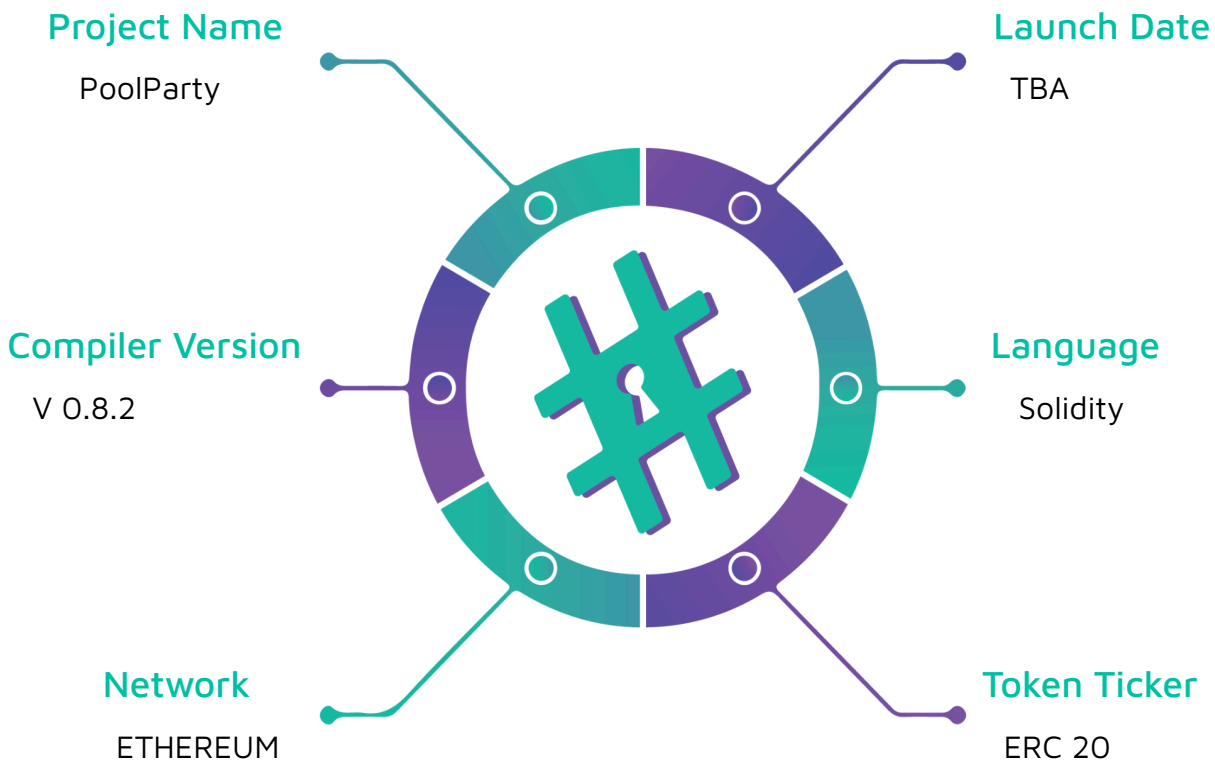
The PoolParty team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts were secure.

Project Context

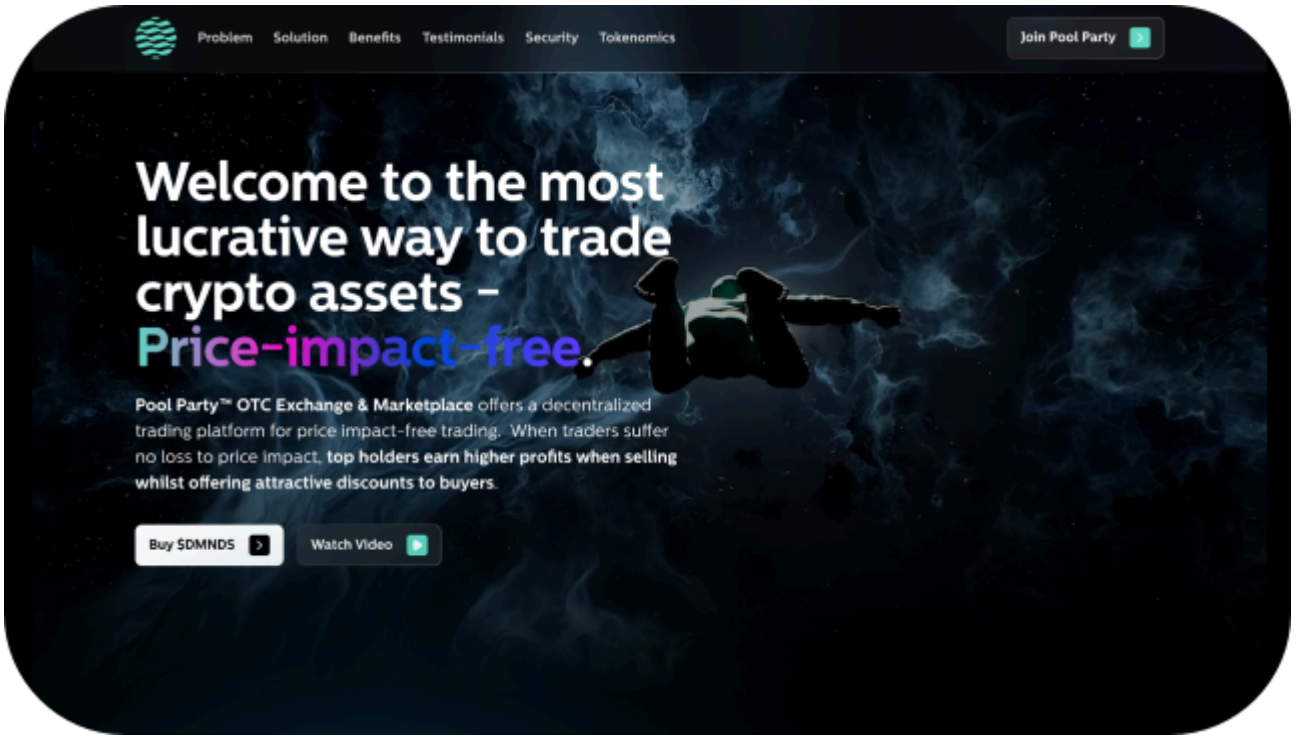
Pool Party Exchange is a decentralized peer-to-peer marketplace and exchange that facilitates secure and efficient over-the-counter (OTC) token swaps. The platform offers users the ability to trade a variety of cryptocurrencies in a way that circumvents the primary liquidity pool so large holders can sell without price impact, and buyers can acquire below-market priced tokens. Pool Party Exchange emphasizes transparency and user control, leveraging blockchain technology to provide a decentralized exchange experience.

Project Name: PoolParty
Compiler Version: 0.8.20
Website: www.poolparty.market
Logo:

Visualised Context:



Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the PoolParty project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**PoolParty Protocol Smart Contracts**
Platform	**Ethereum / Solidity**
Audit Date	**May, 2024**
Contract 1	PoolParty.sol
Contract 1 MD5 Hash	05a69dbb478eefc22c6b85cddc13e83f
Contract 2	resellerCommunityReward.sol
Contract 2 MD5 Hash	4d64c3efda216570ce99d0821238f4b354eb3ff6
Contract 3	publicPoolContract.sol
Contract 3 MD5 Hash	caf38f9642a2ce6923ce7e1cde39849d
Contract 4	publicPool.sol
Contract 4 MD5 Hash	a9867549e82c43d5a87160ef66b581de
Contract 5	ownedPoolContract.sol
Contract 5 MD5 Hash	a21960c92ffc3f37d6cd0be940d7435a
Contract 6	ownedPool.sol
Contract 6 MD5 Hash	79385965a0630259aaaedd3081819c58
Contract 7	updateDiamondStruct.sol
Contract 7 MD5 Hash	8f4ddeb60fc3677aba1d4c8ce3ce8188
Contract 8	diamondStructs.sol
Contract 8 MD5 Hash	93742321de2004524178ac9340757af0

Security Rating

After Hashlock's Audit, we found the smart contracts to be "Secure". The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some significant vulnerabilities that have since been addressed by the PoolParty team.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section.

We initially identified some significant vulnerabilities that have since been addressed

Hashlock found and the PoolParty team resolved:

3 High-severity vulnerabilities

4 Medium-severity vulnerabilities

2 Low-severity vulnerabilities

6 Gas Optimisations

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Behaviours

Claimed Behaviour	Actual Behaviour
PoolParty.sol - Initializes contract with roles and platform information - Grants admin roles to deployer and contract - Updates PoolParty fee and receiver - Verifies projects and resellers - Links Twitter handles to user addresses - Locks/unlocks users and withdraws locked funds - Claims and checks vested tokens - Creates and manages owned and public pools - Updates pool data including vesting, fixed prices, and visibility - Contributes to pools and buys from pools - Ensures safe transfer of tokens and Ether - Emits events for pool creation, updates, verification, and token claims	**Contract achieves this functionality.**
resellerCommunityRewards.sol - Initializes contract with CARATS, stakingContract, and PoolParty addresses - Uses SafeERC20 library for secure token operations - Allows contract to receive Ether - Manages shareholding by adding/removing shareholders and tracking their amounts - Accepts deposits in CARATS tokens from shareholders - Processes withdrawals for shareholders, updating their share amounts - Distributes rewards from the contract's Ether balance to shareholders - Calls PoolParty contract to deposit rewards for shareholders - Provides a function to get the total count of shareholders	**Contract achieves this functionality.**

publicPoolContract.sol - Inherits from Initializable, AccessControlUpgradeable, and UUPSUpgradeable for upgradeable contracts - Defines a constant UPGRADER_ROLE for access control - Stores the address of the diamond interface - Initializes the contract, setting up roles for access control and upgrades - Authorizes upgrades only for addresses with the UPGRADER_ROLE - Allows creation of public pools, requiring a non-zero token address and setting up the pool with token amount, diamond interface, and owner - Updates the diamond interface address and transfers admin role to the new interface while revoking it from the sender	
publicPool.sol - Inherits from ERC20, AccessControl, and ReentrancyGuard. - Utilizes SafeERC20 for safe token operations. - Declares various contract addresses and variables related to token balance, fees, and contributors. - Defines a constant role DIAMOND_ADMIN for access control. - Constructor initializes the contract with token address, amount, diamond interface, and owner. - Constructor sets initial values and grants roles. - Receive function allows the contract to receive ETH. - `diamondTransfer` function handles token transfer, fee calculation, and ETH distribution. - `checkAmounts` validates the price and amount using the spot aggregator. - `checkFeeData` fetches and sets fee-related data from the PoolParty interface. - `takeFee` calculates and distributes fees among various receivers.	

- `contribute` allows users to contribute tokens to the pool, updating internal state variables. - `getPlaceInLine` returns the position of a user in the contributors' list. - `updateOwner` function is intentionally designed to always revert as public pools cannot be owned. - `cancelPool` allows contributors to withdraw their tokens, hiding the pool if the balance becomes zero.	
ownedPoolContract.sol - Inherits from Initializable, AccessControlUpgradeable, and UUPSUpgradeable. - Declares a constant UPGRADER_ROLE for access control. - Declares a variable `diamondInterface` for the diamond interface address. - `initialize` function sets up roles and initializes the contract. - `_authorizeUpgrade` function ensures only users with the UPGRADER_ROLE can upgrade the contract. - `createOwnedPool` function creates a new owned pool contract with specified parameters. - `updateInterfaceAddress` function updates the diamond interface address, grants DEFAULT_ADMIN_ROLE to the new interface, and revokes it from the sender.	
ownedPool.sol - `diamondTransfer`: Handles token transfers, applying fees, and updating balances. - `checkAmounts`: Validates the price and amount of tokens being transferred. - `checkFeeData`: Retrieves fee data from the `PoolParty` contract. - `updateBalances`: Updates the balances of the pool, buyer, and handles reservations. - `takeFee`: Calculates and distributes various fees. - `claimVestedTokens`: Allows users to claim their vested tokens.	

- `contribute`: Allows the pool owner to contribute more tokens to the pool. - `updateVested`: Updates vesting status and initial distribution percentage. - `updateOwner`: Transfers pool ownership and potentially prevents cancellation. - `cancelPool`: Cancels the pool and transfers remaining tokens back to the owner. - `preventCancellation`: Toggles the pool's ability to be canceled. - `updateAmounts`: Updates public and specific token amounts and pool type.	
updateDiamonstruct.sol - Tracks changes in reserved token amounts based on different conditions like Twitter purchases or regular purchases. - Manages changes in public sale amounts and updates user and buyer information accordingly. - Records buyer and seller information when a purchase is made. - Updates user and Twitter reserved pool information. - Manages the creation and update of pool structures, including ownership, discounts, and public sale amounts. - Handles the transfer of ownership of a pool from one user to another. - Manages vested token claims and checks for vested token amounts. - Calculates the remaining public token amounts after deducting reserved amounts. - Handles the verification of projects and resellers, including fees and rewards. - Manages the updating of vesting data for users. - Manages the updating of vesting information for pools. - Handles the cancellation of pools from public sets.	

Code Quality

This Audit scope involves the smart contracts of the PoolParty project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring is required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the PoolParty projects smart contract code in the form of zip file

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments help understand the overall architecture of the protocol.

Dependencies

Per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry-standard open-source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues and inefficiencies

Audit Findings

Medium

[M-01] PoolParty#receive - Mistakenly sent ETH will be Stuck

Description

The PoolParty contracts have a receive() function without any withdrawal mechanism. This can lead to a scenario where any Ether (ETH) sent to the contract address becomes permanently locked within the contract, as there is no method to transfer or withdraw these funds.

Vulnerability Details

According to Slither analyzer detector documentation

Possible functions that receive funds with the payable attribute must have a withdraw function to secure that funds can be sent out from the function or remove the payable attribute.

The PoolParty contracts allows the reception of Ether through the standard receive() external payable function, which is designed to accept Ether transactions. However, the lack of a corresponding withdrawal function within the contract means that any received Ether cannot be redistributed or reclaimed, essentially becoming stuck within the contract's balance.

So, if a user mistakenly sends eth to the contract then it will be stuck and he won't be able to withdraw it

```
Receive() external payable {

}
```

Impact

Loss of User's Eth

Recommendation

Implement a secure withdrawal function that allows the contract owner or another authorized party to transfer Ether out of the FeeSplitter contract.

Note:

PoolParty decided to leave this as is. If users send ETH incorrectly to the pool contracts, it will be forfeited. The team believes that adding a withdrawal function could introduce unnecessary centralization risks.

Status

Acknowledged

[M-02] publicPool#takeFee - If any of the Fee receiver is malicious then the function could revert

Description

If any of the Fee receivers is malicious then the function could revert

Vulnerability Details

The function takeFee sends ETH to ProjectFeeReceiver, ResellerFeeReceiver, and PoolPartyFeeReceiver but the issue is that if any of the receivers is malicious or a contract that has some gas utilisation computation in the receive/fallback function then the function could revert and create DOS permanently

```
if(isVerified && isReseller) {

            (success, ) = payable(ProjectFeeReceiver).call{value: projectFee}("");

            if(!success) {

                diamondFee += projectFee;

            }

            (success, ) = payable(ResellerFeeReceiver).call{value: resellerFee}("");
```

```
            if(!success) {
                diamondFee += resellerFee;
            }
            (success, ) = payable(PoolPartyFeeReceiver).call{value: diamondFee}("");
            require(success, "Failed to send Diamond Fee");
            return newPrice;
```

Impact

Permanent DOS of the function

Recommendation

It is recommended that instead of sending funds implement a pull strategy where the receivers will pull their funds from the smart contract

Note:

All fee receivers will be thoroughly vetted by the PoolParty admin team before being approved for listing. The team chose not to implement the pull function due to contract size constraints.

Status

Acknowledged

Low

[L-01] publicPool#updateOwner - function will always revert

Description

updateOwner function will always revert cause there isn't any other code in this function

Recommendation

Make sure to add relevant code to update the owner

Note:

This behavior is intentional. Any updateOwner call to a public pool is meant to revert since public pools do not have owners.

Status

Acknowledged

[L-02] resellerCommunityRewards - Use Ownable2Step instead of Ownable

Description

By using Ownable2Step or Ownable2StepUpgradeable instead of the standard `Ownable` contract, you add an additional layer of security to your smart contracts. These contracts require the new owner to actively accept the ownership transfer, reducing the risk of mistakenly transferring the ownership to the wrong address.

Recommendation

Make sure to use Ownable2step

Status

Acknowledged

Gas

[G-03] Contracts - Using bools for storage incurs overhead

Description

Use uint256(1) and uint256(2) for true/false to avoid a Gwarmaccess (100 gas), and to avoid Gsset (20000 gas) when changing from 'false' to 'true', after having been 'true' in the past. See source.

Status

Acknowledged

[G-05] Contracts - Use Custom Errors

Description

Instead of using error strings, to reduce deployment and runtime cost, you should use Custom Errors. This would save both deployment and runtime cost. Source

Status

Acknowledged

[G-06] Contracts - Using private rather than public for constants, saves gas

Description

If needed, the values can be read from the verified contract source code, or if there are multiple values there can be a single getter function that returns a tuple of the values of all currently-public constants. Saves 3406-3606 gas in deployment gas due to the compiler not having to create non-payable getter functions for deployment calldata, not having to store the bytes of the value outside of where it's used, and not adding another entry to the method ID table

Status

Acknowledged

Centralisation

The PoolParty project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlocks analysis, the PoolParty project seems to have a sound and well-tested code base, now that our findings have been resolved or acknowledged to achieve security. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

Due to the fact that the total number of test cases are unlimited, the audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds, and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au


#Hashlock.